

DSA STUDY BLUEPRINT SERIES

38. Permutations Array String Backtracking

Generating Unique Permutations of Array/String

Section 06 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Permutations:** Arrangement permutations of list elements (size $N!$).
- Track elements using swap swaps or boolean arrays.

Memory & Execution Blueprint

"abc" $\Rightarrow$ Swap swaps $\Rightarrow$ "acb", "bac", "bca", "cab",
"cba"

C++ Implementation Reference

Standard code layout and syntax

```
void permute(vector& nums, int idx, vector<vector>& ans) {  
    if (idx == nums.size()) { ans.push_back(nums); return; }  
    for (int i = idx; i < nums.size(); i++) {  
        swap(nums[idx], nums[i]);  
        permute(nums, idx + 1, ans);  
        swap(nums[idx], nums[i]); // Backtrack  
    }  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N * N!)$
Space Complexity	$O(N)$

Key Practices & Gotchas

- **Important:** If input has duplicates, skip identical swaps to avoid duplicate permutations.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace recursive decision tree transitions generating subsets for [1, 2] :

Decision Level	Element choice	Recursive action	Subsets array state
i = 0	Include 1	Recurse level 1	[[1]]
i = 1	Exclude 2	Base case hit	Add subset [1] to results

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **State Reversals:** Essential backtrack actions pop choice selections to restore node states before adjacent searches.
- **Pruning:** Skip duplicate recursive paths by checking elements duplicates before branching.
- **Related LeetCode Problems:** #78 Subsets, #90 Subsets II, #46 Permutations.